

Linear data structures in Go

Contents

1. [Already covered elsewhere](#)
2. [Linked list](#)
3. [Stack](#)
4. [Queue](#)
5. [Deque \(double-ended queue\)](#)
6. [Ring buffer](#)
7. [Channels as concurrent linear structures](#)
8. [Choosing the right structure](#)
9. [Performance notes](#)
10. [Best practices](#)
11. [Common gotchas](#)
12. [Quick reference](#)

Sequential containers: lists, stacks, queues, dequeues, ring buffers. The ones with built-in support, the ones in `container/`, and the patterns you build by hand.

1. Already covered elsewhere

- **Arrays** - fixed-size, value-semantic. See `go-basics.pdf` and `vars-types-collections.pdf`.
- **Slices** - dynamic, reference-semantic, the workhorse. See `slices.pdf` and `slice-internals.pdf`.
- **Strings** - immutable byte sequence. See `go-basics.pdf`.
- **Maps** - hash table. See `map-internals.pdf`.

This sheet covers everything else: linked lists, stacks, queues, dequeues, ring buffers.

2. Linked list

A chain of nodes where each holds a value and a pointer to the next. Use when:

- Constant-time insertion and deletion at arbitrary positions matter.
- The slice's contiguous memory hurts (e.g., huge elements, frequent middle inserts).

In practice, slices win 95% of the time even when a linked list seems theoretically better. Linked lists are cache-unfriendly; modern CPUs hate them.

Singly linked list (hand-rolled)

```
type Node[T any] struct {
    Value T
    Next *Node[T]
}

type LinkedList[T any] struct {
    head *Node[T]
    size int
}

func (l *LinkedList[T]) PushFront(v T) {
    l.head = &Node[T]{Value: v, Next: l.head}
    l.size++
}

func (l *LinkedList[T]) PopFront() (T, bool) {
    var zero T
    if l.head == nil { return zero, false }
    v := l.head.Value
    l.head = l.head.Next
    l.size--
    return v, true
}

func (l *LinkedList[T]) Len() int { return l.size }

func (l *LinkedList[T]) Each(f func(T)) {
    for n := l.head; n != nil; n = n.Next {
        f(n.Value)
    }
}
```

Operations:

Op	Complexity
PushFront	O(1)
PopFront	O(1)
PushBack	O(n) without tail pointer; O(1) with
Find	O(n)
Delete	O(n) to find; O(1) once located (with prev pointer)

Doubly linked list (hand-rolled)

```
type DNode[T any] struct {
    Value T
    Prev *DNode[T]
    Next  *DNode[T]
}

type DLinkedList[T any] struct {
    head, tail *DNode[T]
    size       int
}

func (l *DLinkedList[T]) PushBack(v T) {
    n := &DNode[T]{Value: v, Prev: l.tail}
    if l.tail == nil {
        l.head = n
    } else {
        l.tail.Next = n
    }
    l.tail = n
    l.size++
}

func (l *DLinkedList[T]) Remove(n *DNode[T]) {
    if n.Prev != nil { n.Prev.Next = n.Next } else { l.head = n.Next }
    if n.Next != nil { n.Next.Prev = n.Prev } else { l.tail = n.Prev }
    l.size--
}
```

Doubly linked lists shine for:

- LRU caches (need $O(1)$ move-to-front).
- Bidirectional iteration.
- Removing a known node in $O(1)$.

container/list (stdlib doubly linked list)

The standard library has a generic-free doubly linked list. Values are stored as `any`.

```
import "container/list"

l := list.New()
l.PushBack(1)
l.PushBack(2)
l.PushFront(0)

for e := l.Front(); e != nil; e = e.Next() {
    fmt.Println(e.Value)      // 0, 1, 2
}

mid := l.PushBack(99)
l.Remove(mid)
```

Operations:

Method	What it does
PushFront(v)	Insert at head; returns *Element
PushBack(v)	Insert at tail
InsertBefore(v, mark)	Insert before a known element
InsertAfter(v, mark)	Insert after a known element
Remove(e)	Remove an element
MoveToFront(e)	O(1) reorder
MoveToBack(e)	O(1) reorder
Front() / Back()	First / last element
Len()	Size

The boxing into `any` costs allocations per insertion. For generic-friendly lists, write your own (above) or use a third-party library.

When to use a linked list

Yes	No
LRU cache (need O(1) move-to-front + remove)	"I want a list" -> use a slice
Free-list allocators	Anywhere cache locality matters
Custom intrusive structures	Numeric or small-element data
Bidirectional iteration with frequent removes	Random access

For most "list-like" needs, a slice beats a linked list. Trust the benchmark, not the textbook.

3. Stack

LIFO: last in, first out. The simplest structure to build on a slice.

```
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    n := len(s.items)
    if n == 0 { return zero, false }
    v := s.items[n-1]
    s.items[n-1] = zero           // help GC for pointer types
    s.items = s.items[:n-1]
    return v, true
}

func (s *Stack[T]) Peek() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    return s.items[len(s.items)-1], true
}

func (s *Stack[T]) Len() int { return len(s.items) }
```

The “zero the slot” line matters when T holds pointers. Otherwise the popped value stays reachable and the GC can’t reclaim it.

Operations:

Op	Complexity
Push	O(1) amortized
Pop	O(1)
Peek	O(1)
Len	O(1)

Common uses:

- Function call traces, parser state.
- Undo/redo histories.
- Depth-first search (explicit stack instead of recursion).
- Expression evaluation (operator/operand stacks).
- Matching brackets, validating nesting.

4. Queue

FIFO: first in, first out. Less obvious than stacks because naive “slice queue” is $O(n)$ on dequeue.

Naive slice queue (slow)

```
queue = append(queue, x)           // enqueue: O(1) amortized
x = queue[0]; queue = queue[1:]    // dequeue: O(1) per call, BUT
                                   // memory never reclaimed; backing array grows
```

The backing array isn't released as you `queue[1:]`. After 1 million enqueues + dequeues, the array still holds the high-water-mark allocation. For long-lived queues, use one of the patterns below.

Slice queue with periodic compaction

```
type Queue[T any] struct {
    items []T
    head  int
}

func (q *Queue[T]) Push(v T) { q.items = append(q.items, v) }

func (q *Queue[T]) Pop() (T, bool) {
    var zero T
    if q.head == len(q.items) { return zero, false }
    v := q.items[q.head]
    q.items[q.head] = zero
    q.head++
    if q.head > 64 && q.head*2 > len(q.items) {
        q.items = append(q.items[:0], q.items[q.head:]...)
        q.head = 0
    }
    return v, true
}

func (q *Queue[T]) Len() int { return len(q.items) - q.head }
```

When head advances past half the slice, copy the remainder back to the front. Amortized $O(1)$ per operation.

Ring buffer queue (fixed capacity)

```
type RingQueue[T any] struct {
    buf      []T
    head, tail int
    size      int
}

func NewRingQueue[T any](cap int) *RingQueue[T] {
    return &RingQueue[T]{buf: make([]T, cap)}
}

func (q *RingQueue[T]) Push(v T) bool {
    if q.size == len(q.buf) { return false }           // full
    q.buf[q.tail] = v
    q.tail = (q.tail + 1) % len(q.buf)
    q.size++
    return true
}

func (q *RingQueue[T]) Pop() (T, bool) {
    var zero T
    if q.size == 0 { return zero, false }
    v := q.buf[q.head]
    q.buf[q.head] = zero
    q.head = (q.head + 1) % len(q.buf)
    q.size--
    return v, true
}

func (q *RingQueue[T]) Len() int { return q.size }
```

Fixed cap, no allocation per operation, predictable memory. Use when:

- You know the upper bound.
- You're in a hot path.
- Latency must be predictable.

Linked list queue

```
import "container/list"

q := list.New()
q.PushBack(x)           // enqueue
e := q.Front()           // peek
q.Remove(e)              // dequeue
```

Each enqueue allocates. Use only when capacity is unbounded and growth is slow.

Channel as a queue

In concurrent code, a buffered channel is a thread-safe queue with built-in blocking semantics.

```
ch := make(chan T, 100)
ch <- x                                // enqueue (blocks if full)
v := <-ch                             // dequeue (blocks if empty)
```

Best when producers and consumers are different goroutines. Not for single-goroutine queues; the synchronization overhead is wasted.

Common uses:

- Task scheduling.
- BFS (queue of nodes to explore).
- Producer/consumer pipelines.
- Rate limiting (token queue).
- Message buffering.

5. Deque (double-ended queue)

Add and remove from both ends in $O(1)$. Useful for sliding-window algorithms, work-stealing schedulers, monotonic queues.

Doubly linked list deque

```
type Deque[T any] struct {  
    l *list.List  
}  
  
func NewDeque[T any]() *Deque[T] { return &Deque[T]{l: list.New()} }  
  
func (d *Deque[T]) PushFront(v T) { d.l.PushFront(v) }  
func (d *Deque[T]) PushBack(v T) { d.l.PushBack(v) }  
  
func (d *Deque[T]) PopFront() (T, bool) {  
    var zero T  
    e := d.l.Front()  
    if e == nil { return zero, false }  
    d.l.Remove(e)  
    return e.Value.(T), true  
}  
  
func (d *Deque[T]) PopBack() (T, bool) {  
    var zero T  
    e := d.l.Back()  
    if e == nil { return zero, false }  
    d.l.Remove(e)  
    return e.Value.(T), true  
}
```

Circular buffer deque

For cache-friendly performance, a ring-buffer deque:

```

type CDeque[T any] struct {
    buf      []T
    head, tail int
    size      int
}

func (d *CDeque[T]) grow() {
    n := len(d.buf) * 2
    if n == 0 { n = 8 }
    next := make([]T, n)
    for i := 0; i < d.size; i++ {
        next[i] = d.buf[(d.head+i)%len(d.buf)]
    }
    d.buf = next
    d.head = 0
    d.tail = d.size
}

func (d *CDeque[T]) PushBack(v T) {
    if d.size == len(d.buf) { d.grow() }
    d.buf[d.tail] = v
    d.tail = (d.tail + 1) % len(d.buf)
    d.size++
}

func (d *CDeque[T]) PushFront(v T) {
    if d.size == len(d.buf) { d.grow() }
    d.head = (d.head - 1 + len(d.buf)) % len(d.buf)
    d.buf[d.head] = v
    d.size++
}

```

PopFront and PopBack mirror these.

Common uses:

- Sliding window max/min (monotonic deque).
- Work-stealing job queues.
- Browser back/forward history.
- Some BFS variants.

6. Ring buffer

A fixed-size circular buffer. Used for log streams, sample buffers, and bounded queues. Same shape as the ring-queue above; just named differently when the use is “rolling window” instead of “queue.”

container/ring (stdlib)

The stdlib ring is unusual: it’s a doubly-linked circular list, not a contiguous buffer. Rarely the right tool, but available.

```

import "container/ring"

r := ring.New(5)                                // ring of length 5
for i := 0; i < 5; i++ {
    r.Value = i
    r = r.Next()
}

r.Do(func(v any) { fmt.Println(v) })             // walks once around: 0 1 2 3 4

```

Use cases are narrow (round-robin lists, fixed-size cyclic structures). For most rolling buffers, a contiguous array with modular indexing wins.

Custom contiguous ring buffer

```

type Ring[T any] struct {
    buf []T
    head int
    size int
}

func NewRing[T any](cap int) *Ring[T] {
    return &Ring[T]{buf: make([]T, cap)}
}

// Add to ring, overwriting the oldest if full
func (r *Ring[T]) Add(v T) {
    if r.size < len(r.buf) {
        r.buf[(r.head+r.size)%len(r.buf)] = v
        r.size++
    } else {
        r.buf[r.head] = v
        r.head = (r.head + 1) % len(r.buf)
    }
}

// Snapshot the current contents in order
func (r *Ring[T]) Snapshot() []T {
    out := make([]T, r.size)
    for i := 0; i < r.size; i++ {
        out[i] = r.buf[(r.head+i)%len(r.buf)]
    }
    return out
}

```

Common uses:

- Recent-N event log (last 1000 errors).
- Audio/video sample buffers.
- Bounded telemetry buffers before flushing.

- Rate limiter sliding window (timestamps).

7. Channels as concurrent linear structures

A buffered channel is a thread-safe FIFO queue with blocking. An unbuffered channel is a rendezvous point.

```
// bounded thread-safe queue
ch := make(chan T, 1000)

// enqueue (blocks when full)
ch <- v

// non-blocking enqueue
select {
case ch <- v:
default:
    // dropped: queue full
}

// dequeue
v := <-ch

// non-blocking dequeue
select {
case v := <-ch:
    use(v)
default:
    // empty
}
```

Properties:

- Send and receive are linearizable (atomic from any goroutine's view).
- Send on a closed channel panics.
- Receive on a closed channel returns the zero value with `ok=false`.
- A nil channel blocks forever (useful trick).

Channels are the right answer when producers and consumers are different goroutines. They're the wrong answer when one goroutine owns the queue (use a plain slice or ring buffer).

8. Choosing the right structure

Need	Pick
Dynamic ordered list, mostly append + random access	Slice
Stack	Slice
Single-goroutine queue (small, bounded)	Ring buffer

Need	Pick
Single-goroutine queue (unbounded)	Slice + compaction, or container/list
Multi-goroutine queue	Buffered channel
LRU cache	Doubly linked list + map
Bidirectional iteration with frequent middle remove	Doubly linked list
Sliding window with both-end ops	Deque (circular buffer)
Last-N events	Ring buffer (contiguous)
Round-robin small list	container/ring
BFS frontier	Slice queue or container/list

9. Performance notes

A few benchmarks worth knowing:

- Slice-based stack: a couple of nanoseconds per Push/Pop. Hard to beat.
- container/list adds about 30-50 ns per operation due to allocation + boxing into `any`.
- Buffered channel send/receive in the same goroutine: 30-100 ns; across goroutines: 100-500 ns including scheduling.
- A custom ring buffer beats container/ring and container/list for raw throughput by a wide margin.

Use slices first. Switch to a custom structure when profiling shows specific costs (boxing, copying, contention) you can address.

10. Best practices

1. Default to slices. They cover stacks, simple queues, and most “list” needs better than dedicated structures.
2. Watch for slice growth. Long-running “slice as queue” with `queue[1:]` leaks memory until you compact.
3. Zero out popped slots that contain pointers, so the GC can reclaim them.
4. Use channels for cross-goroutine queues. Don’t roll a mutex + slice unless you have a specific reason.
5. Use ring buffers for bounded, hot-path queues. No allocations per operation.
6. Avoid container/list for performance-sensitive code. The `any` boxing is real cost.
7. For LRU-style structures, pair a doubly linked list with a map. Lookup by key in the map; reorder in the list.
8. Document blocking semantics. Channel queues block by default; explicit select default for non-blocking ops.

11. Common gotchas

Gotcha	Symptom	Fix
<code>queue = queue[1:]</code> for long-lived queue	Backing array never freed	Compact periodically or use ring buffer
Popped slice slot keeps pointer alive	Memory leak	Zero the slot before shrinking
container/list for hot path	Allocations everywhere	Custom linked list or slice
Linked list when you wanted random access	O(n) instead of O(1)	Use a slice
Unbounded channel queue	Goroutine memory grows	Set a sensible buffer size
Forgetting ok on channel receive	Reading zero from closed channel as data	Use <code>v, ok := <-ch</code>
Stack peek returns stale data after pop	Reference held in slice	Zero out on pop
Ring buffer overflow silently dropped	Lost messages	Return bool from Push; let caller decide
Manually implementing thread-safe queue	Race conditions	Use a channel
Wrong end of deque	Reversed iteration order	Pick PushFront vs PushBack carefully

12. Quick reference

Structure	Best Go implementation
Stack	Slice
FIFO queue (single goroutine, bounded)	Custom ring buffer
FIFO queue (single goroutine, unbounded)	Slice with periodic compaction
FIFO queue (multi-goroutine)	Buffered channel
Doubly linked list	container/list or hand-rolled generic
Deque	Circular buffer (ring)
Rolling window	Ring buffer with overwrite
Round-robin small list	container/ring
LRU primitive	Doubly linked list + map
Sliding-window stats	Ring buffer of timestamps
Push at both ends	Deque or circular buffer

Structure	Best Go implementation
Bounded message broker	Buffered channel
Lock-free queue	Specialized library (e.g., chenzhuoyu/iasm queue, lock-free MPMC libraries)